УГНТУ • ЦЕНТР ТОП ИТ • 09.03.03 ПРИКЛАДНАЯ ИНФОРМАТИКА

Программирование и прототипирование ИТ-решений

Циклические конструкции: for, while, break/continue/else, итераторы, range/enumerate/zip

ЛЕКЦИЯ 5

Ахметов Ильнур Вазирович

План лекции

1. Цикл for: итерация по последовательностям, range().
2. Цикл while: повторение по условию.
3. break, continue и else в циклах.
4. enumerate() и zip(): параллельная итерация.
5. Итераторы, генераторы списков и выражений.
6. Практикумы и кейсы из нефтегазовой отрасли.

В конце — вопросы для закрепления (по одному на слайд).

ПОВТОРЕНИЕ

Что мы знаем из лекции 4

- if-elif-else — программа учится принимать решения.
- Логические операторы: and, or, not.
- match-case — сопоставление с шаблоном.
- Валидация: проверка типа, диапазона, бизнес-правил.
- Но программа всё ещё выполняется один раз — сверху вниз.
- Сегодня: заставим программу ПОВТОРЯТЬ действия.

консоль Python

```
>>> # линейно: одно действие — один проход
>>> print('Скважина К-12')
>>> print('Скважина К-13')
>>> print('Скважина К-14')

>>> # а если скважин 100?
>>> # писать 100 строк print()?
>>> # нужен ЦИКЛ!
```

Что такое цикл и зачем он нужен

- Цикл — это повторение блока кода несколько раз.
- Два вида циклов в Python:
- for — повторение по известной последовательности.
- while — повторение, пока условие истинно.
- Цикл = «делай это N раз» или «делай, пока...».
- Без циклов нельзя обработать массивы, файлы, ряды данных.

код

```
# вместо 100 строк:
for i in range(100):
    print(f"Скважина №{i+1}")

# одна конструкция — 100 повторений
# меняется только i: 0, 1, 2, ..., 99
```

for: синтаксис и итерация

- for <переменная> in <последовательность>:
- <блок> — 4 пробела (как в if!)
- Переменная принимает каждый элемент последовательности по очереди.
- Последовательность: список, строка, range, словарь, файл...
- После цикла переменная остаётся в памяти (последний элемент).

код

```
wells = ['К-12', 'К-13', 'К-14']

for well in wells:
    print(f"Скважина {well}")

# Вывод:
# Скважина К-12
# Скважина К-13
# Скважина К-14
```

for по строке: посимвольная обработка

- Строка — тоже последовательность (символов).
- for пройдёт по каждому символу.
- Удобно для подсчёта, фильтрации, анализа кода.
- Можно собрать новую строку через накопление.

код

```
code = 'К-12-Д'
digits = 0

for ch in code:
    if ch.isdigit():
        digits += 1

print(f"Цифр в коде: {digits}")
# Цифр в коде: 2
```

for по списку словарей: реестр скважин

- Самый частый паттерн: итерация по списку объектов.
- Каждый элемент — словарь; обращаемся по ключам.
- Внутри цикла можно агрегировать, фильтровать, считать.
- Это основа любой обработки данных в Python.

код

```
wells = [
    {"id": "К-12", "дебит": 85.4},
    {"id": "К-13", "дебит": 0.0},
    {"id": "К-14", "дебит": 12.7},
]

total = 0
for w in wells:
    total += w["дебит"]

print(f"Суммарный дебит: {total:.1f} т/сут")
```

range(): генератор числовых последовательностей

- range(stop) — числа от 0 до stop-1.
- range(start, stop) — от start до stop-1.
- range(start, stop, step) — с шагом step.
- range НЕ создаёт список — это ленивый объект!
- list(range(5)) → [0, 1, 2, 3, 4] — чтобы увидеть.
- Отличие от срезов: range(stop) стартует с 0.

консоль Python

```
>>> list(range(5))
[0, 1, 2, 3, 4]

>>> list(range(1, 6))
[1, 2, 3, 4, 5]

>>> list(range(0, 20, 5))
[0, 5, 10, 15]

>>> list(range(10, 0, -2))   # обратный
[10, 8, 6, 4, 2]
```

Практика: дебит по месяцам через range()

- Задача: вывести дебит скважины по месяцам 1–12.
- Используем range(1, 13) — месяцы с 1 до 12.
- Дебит падает на 2% каждый месяц (естественное снижение).
- Это типичная задача прогноза добычи.

код

```
initial = 100.0  # т/сут
decline = 0.02   # 2% в месяц

for month in range(1, 13):
    debit = initial * (1 - decline) ** month
    print(f"Месяц {month:2d}: {debit:.1f} т/сут")

# Месяц  1:  98.0 т/сут
# Месяц  2:  96.0 т/сут
# ...
# Месяц 12:  78.5 т/сут
```

Накопление результатов в цикле

- Паттерн «аккумулятор»: заводим переменную до цикла.
- Внутри цикла — обновляем её на каждой итерации.
- Сумма, максимум, минимум, счётчик — всё через аккумулятор.
- Не забудьте начальное значение!

код

```
debits = [85.4, 0.0, 12.7, 65.3, 45.1]

total = 0          # аккумулятор-сумма
maximum = debits[0]  # аккумулятор-максимум
count_active = 0   # аккумулятор-счётчик

for d in debits:
    total += d
    if d > maximum: maximum = d
    if d > 0: count_active += 1

print(f"Сумма: {total}, Max: {maximum}, Активных:
{count_active}")
```

Практика: поиск максимального дебита

- Задача: найти скважину с максимальным дебитом.
- Аккумулятор: max_debit и max_well.
- Обновляем при нахождении большего значения.
- Это базовый алгоритм поиска экстремума.

код

```
wells = [
    {"id": "К-12", "дебит": 85.4},
    {"id": "К-13", "дебит": 120.5},
    {"id": "К-14", "дебит": 12.7},
]

max_well = wells[0]["id"]
max_debit = wells[0]["дебит"]

for w in wells:
    if w["дебит"] > max_debit:
        max_debit = w["дебит"]
        max_well = w["id"]

print(f"Лидер: {max_well} ({max_debit} т/сут)")
```

Практика: фильтрация скважин по критерию

- Задача: выбрать скважины с обводнённостью > 80%.
- Аккумулятор: список high_water = [].
- Условие внутри цикла решает — добавить или нет.
- Это базовый паттерн фильтрации данных.

код

```
wells = [
    {"id": "К-12", "обводнённость": 0.24},
    {"id": "К-13", "обводнённость": 0.88},
    {"id": "К-14", "обводнённость": 0.91},
]

high_water = []

for w in wells:
    if w["обводнённость"] > 0.80:
        high_water.append(w)

print(f"Высокая обводнённость: {len(high_water)}
скв.")
```

while: повторение по условию

- while <условие>:
- <блок> — выполняется, пока условие True.
- Условие проверяется ПЕРЕД каждой итерацией.
- Если условие сразу False — блок не выполнится ни разу.
- ОПАСНОСТЬ: если условие никогда не станет False — бесконечный цикл!

код

```
pressure = 30.0  # МПа, падает

while pressure > 5.0:
    print(f"P = {pressure:.1f} МПа — качаем")
    pressure -= 2.0   # падение на 2 МПа

print("Давление критически низкое — стоп")

# P = 30.0 ... P = 6.0 — качаем
# Давление критически низкое — стоп
```

while: накопление суммы

- Задача: суммировать числа 1 + 2 + ... + N.
- Счётчик i увеличиваем на 1, сумму накапливаем.
- Цикл идёт, пока i <= N.
- Классический пример «while + аккумулятор».

код

```
N = 100
i = 1
total = 0

while i <= N:
    total += i
    i += 1       # не забыть! иначе бесконечный
цикл

print(f"Сумма 1..{N} = {total}")
# Сумма 1..100 = 5050
```

Практика: ввод данных до корректного значения

- Самый частый while в реальных программах: повтор ввода.
- Пока пользователь не введёт корректное значение — спрашиваем.
- Это та же валидация из лекции 4, но в цикле.
- Вместо однократной проверки — повтор до успеха.

код

```
while True:
    s = input("Давление, МПа: ").strip()
    if s == "":
        print("Пустой ввод, повторите")
        continue
    try:
        p = float(s)
    except ValueError:
        print("Введите число!")
        continue
    if 0 <= p <= 50:
        break
    print("Давление должно быть [0;50]")
```

Практика: моделирование падения давления

- Задача: смоделировать работу насоса, пока давление в норме.
- На каждом шаге — добыча уменьшает пластовое давление.
- Цикл останавливается, когда давление падает ниже 5 МПа.
- Это типичная симуляция добычи во времени.

код

```
P = 30.0     # пластовое давление
Q = 50.0     # дебит
step = 0

while P >= 5.0:
    step += 1
    print(f"Шаг {step}: P={P:.1f} Q={Q:.1f}")
    P -= 0.5    # давление падает
    Q *= 0.98   # дебит падает на 2%

print(f"Остановлено на шаге {step}")
```

for vs while: когда что использовать

for знаем, сколько раз повторить (или есть готовый список).

while не знаем, сколько раз — повторяем, пока условие истинно.

for идём по элементам последовательности.

while ждём события (ввод, достижение порога, условие).

Правило: если можете написать for — пишите for. while — только когда нужно.

код

```
# for — знаем количество
for month in range(1, 13):
    print(f'Месяц {month}')

# while — не знаем, когда остановиться
while pressure > 5.0:
    pump(pressure)
    pressure -= decline
```

break: досрочный выход из цикла

- break — немедленно прерывает цикл.
- Управление переходит к коду после цикла.
- Используется при поиске: нашли — вышли.
- В сочетании с for — «перебор до первого совпадения».

код

```
wells = ['K-12', 'K-13', 'K-14', 'K-15']

for w in wells:
    print(f"Проверяю {w}...")
    if w == 'K-14':
        print(f"Нашли: {w}")
        break       # дальше не ищем

# Проверяю K-12...
# Проверяю K-13...
# Проверяю K-14...
# Нашли: K-14
```

continue: пропуск итерации

- continue — пропускает остаток текущей итерации.
- Цикл переходит к следующему элементу.
- Удобно для фильтрации: «этот не подходит — следующий».
- Не прерывает цикл целиком (в отличие от break).

код

```
debits = [85.4, 0.0, 12.7, 0.0, 65.3]

for d in debits:
    if d == 0:
        continue    # остановленные пропускаем
    print(f"Дебит: {d:.1f} т/сут")

# Дебит: 85.4 т/сут
# Дебит: 12.7 т/сут
# Дебит: 65.3 т/сут
# (нули пропущены)
```

Практика: поиск первой аномалии

- Задача: найти первую аномалию в ряду телеметрии.
- Норма — значения в [50; 300].
- Нашли аномалию → break (дальше не ищем).
- Дополнительно: пропускаем нули через continue.

код

```
data = [120, 118, 0, 121, 450, 119, 5]

for i, v in enumerate(data):
    if v == 0:
        continue            # пропуск
    if not (50 <= v <= 300):
        print(f"Аномалия: индекс {i}, значение
{v}")
        break               # нашли — выходим

# Аномалия: индекс 4, значение 450
```

else в цикле: «цикл завершился нормально»

- else после цикла — выполняется, если НЕ было break.
- То есть «цикл прошёл до конца, ничего не прерывая».
- Удобно для проверки: «ничего не нашли» — это нормальный вариант.
- Запутывает новичков, но мощный приём.

код

```
wells = ['К-12', 'К-13', 'К-14']
target = 'К-99'   # несуществующая

for w in wells:
    if w == target:
        print(f"Нашли: {w}")
        break
else:
    print(f"{target} не найдена в реестре")

# К-99 не найдена в реестре
```

Вложенные циклы: таблица умножения

- Цикл внутри цикла: для каждого i — пробегаем все j.
- Внешний цикл — строки, внутренний — столбцы.
- Всего итераций: i × j (при 3×3 = 9 проходов).
- Не злоупотребляйте: вложенность > 2 — сложно читать.

код

```
for i in range(1, 4):       # строки
    for j in range(1, 4):   # столбцы
        print(f'{i}x{j}={i*j}', end='  ')
    print()   # перевод строки после каждой
строки

# 1x1=1  1x2=2  1x3=3
# 2x1=2  2x2=4  2x3=6
# 3x1=3  3x2=6  3x3=9
```

Вложенные циклы: матрица дебитов по кустам

- Задача: вывести дебит по кустам и скважинам.
- Внешний цикл — кусты, внутренний — скважины в кусте.
- Внутри — расчёт и форматированный вывод.
- Это двумерная обработка данных — основа табличных расчётов.

код

```
fields = {
    'Куст 3': ['К-12', 'К-13', 'К-14'],
    'Куст 4': ['К-15', 'К-16'],
}

for cluster, wells in fields.items():
    print(f"\n{cluster}:")
    for w in wells:
        print(f"  {w} — в работе")
```

Вложенные циклы: поиск по двум критериям

- Задача: найти скважины с дебитом > 50 И обводнённостью < 30%.
- Внешний цикл — скважины, внутренний — можно не нужен.
- Но если данные — двумерные (кусты × скважины): два уровня.
- break во внутреннем цикле — выходит только из внутреннего!

код

```
for cluster in clusters:
    for well in cluster:
        if well["дебит"] > 50 and
well["обводнённость"] < 0.3:
            print(f"Найдена: {well["id"]}")
            break  # только из внутреннего!
    # внешний цикл продолжится

# Чтобы выйти из обоих — используйте флаг
# или вынесите в функцию с return
```

enumerate(): индекс + значение одновременно

- enumerate(seq) — выдаёт пары (индекс, значение).
- Не нужно писать range(len(seq)) — это антипаттерн!
- Можно задать стартовый индекс: enumerate(seq, start=1).
- Один из самых полезных инструментов Python.

код

```
# ПЛОХО:
for i in range(len(wells)):
    print(i, wells[i])

# ХОРОШО:
for i, w in enumerate(wells):
    print(i, w)

# С нумерацией с 1:
for num, w in enumerate(wells, start=1):
    print(f"{num}. {w}")

# 1. K-12
# 2. K-13
# 3. K-14
```

Практика: нумерация скважин через enumerate()

- Задача: вывести реестр скважин с порядковыми номерами.
- enumerate(wells, start=1) — нумерация с 1.
- Удобно для отчётов, таблиц, вывода пользователю.
- Код читается естественно: «для каждой скважины по номеру...».

код

```
wells = [
    {"id": "К-12", "дебит": 85.4},
    {"id": "К-13", "дебит": 0.0},
    {"id": "К-14", "дебит": 12.7},
]

print(f"{"№":<4}{"Скважина":<10}{"Дебит":>10}")
print("-" * 24)

for num, w in enumerate(wells, start=1):

print(f"{num:<4}{w["id"]:<10}{w["дебит"]:>10.1f}"
)
```

zip(): параллельная итерация по нескольким спискам

- zip(a, b) — выдаёт пары (элемент_a, элемент_b) одновременно.
- Идёт до длины САМОГО КОРОТКОГО списка.
- Можно zip(a, b, c) — тройки.
- Не нужно писать индексы — идём параллельно.

код

```
wells = ['К-12', 'К-13', 'К-14']
debits = [85.4, 0.0, 12.7]
pressures = [18.5, 4.2, 22.0]

for well, d, p in zip(wells, debits, pressures):
    print(f"{well}: дебит={d}, P={p}")

# К-12: дебит=85.4, P=18.5
# К-13: дебит=0.0,  P=4.2
# К-14: дебит=12.7, P=22.0
```

Практика: сводка скважин через zip()

- Задача: вывести сводку из трёх списков — скважины, дебит, давление.
- zip() объединяет их в тройки — одна строка на скважину.
- Если списки разной длины — лишние элементы просто отбрасываются.
- zip(*) — обратная операция: «распаковать» пары обратно в списки.

код

```
ids = ['К-12', 'К-13', 'К-14']
debits = [85.4, 0.0, 12.7]

# сводка с отметкой статуса
for wid, d in zip(ids, debits):
    status = "активна" if d > 0 else
"остановлена"
    print(f"{wid}: {d:.1f} т/сут — {status}")

# К-12: 85.4 т/сут — активна
# К-13: 0.0 т/сут — остановлена
# К-14: 12.7 т/сут — активна
```

Итераторы: что это и как работает for

- for работает с ИТЕРАТОРАМИ — не со списками напрямую.
- Итератор — объект, который выдаёт следующий элемент по запросу.
- iter(seq) — создаёт итератор из последовательности.
- next(it) — берёт следующий элемент; StopIteration — конец.
- for автоматически вызывает iter() и next() под капотом.

консоль Python

```
>>> it = iter([10, 20, 30])
>>> next(it)
10
>>> next(it)
20
>>> next(it)
30
>>> next(it)
Traceback (most recent call last):
  StopIteration
```

Итераторы vs списки: ленивые вычисления

- Список хранит ВСЕ элементы в памяти.
- Итератор хранит только ТЕКУЩИЙ элемент и правило перехода.
- range(1_000_000) — не занимает память на миллион чисел!
- Это позволяет работать с огромными данными и файлами.
- Нельзя дважды пройти один итератор — он одноразовый.

консоль Python

```
>>> r = range(1_000_000)
>>> import sys
>>> sys.getsizeof(r)
48          # 48 байт — независимо от размера!

>>> lst = list(r)
>>> sys.getsizeof(lst)
8000056     # 8 МБ — весь список в памяти

>>> # итератор одноразовый:
>>> it = iter([1, 2, 3])
>>> list(it)  # [1, 2, 3]
>>> list(it)  # [] — уже пуст!
```

Файл — тоже итератор: потоковое чтение

- Открытый файл — итератор строк.
- for line in f: — читает по одной строке, не загружая весь файл.
- Критично для больших журналов логов (гигабайты).
- Это основа лабораторной работы № 9.

код

```
# файл 10 ГБ — в память не поместится!
with open("telemetry.log", encoding="utf-8") as
f:
    for line in f:
        # в памяти — только одна строка
        if 'ERROR' in line:
            print(line.strip())

# та же логика, но весь файл в памяти — ПЛОХО:
# lines = f.readlines()  # 10 ГБ в RAM!
```

Генератор списка (list comprehension)

- [выражение for элемент in последовательность]
- Создаёт новый список в одну строку.
- Можно с условием: [x for x in seq if условие].
- Короче и быстрее, чем цикл + append.
- Не злоупотребляйте сложными выражениями — читаемость важнее.

код

```
# через цикл:
squares = []
for x in range(1, 6):
    squares.append(x ** 2)
# [1, 4, 9, 16, 25]

# через comprehension — одна строка:
squares = [x ** 2 for x in range(1, 6)]

# с условием:
active = [w for w in wells if w['дебит'] > 0]
```

Практика: фильтрация и преобразование данных

- Задача: получить ID активных скважин с высоким дебитом.
- Одна строка вместо цикла + if + append.
- Можно комбинировать: фильтр + преобразование.
- Помните: создаёт НОВЫЙ список — память тратится.

код

```
wells = [
    {"id": "К-12", "дебит": 85.4,
"обводнённость": 0.24},
    {"id": "К-13", "дебит": 0.0,
"обводнённость": 0.0},
    {"id": "К-14", "дебит":
120.5,"обводнённость": 0.88},
]

# ID скважин с дебитом > 50:
high = [w["id"] for w in wells if w["дебит"] >
50]
# ['К-12', 'К-14']

# Дебиты активных, округлённые:
rounded = [round(w["дебит"]) for w in wells if
w["дебит"] > 0]
# [85, 120]
```

Генератор выражений (generator expression)

- (выражение for элемент in последовательность) — скобки () вместо [].
- Не создаёт список — возвращает итератор (лениво).
- Память: O(1) вместо O(n).
- Идеально для sum(), max(), len() — прокатили и забыли.

код

```
# list comprehension — список в памяти:
squares = [x**2 for x in range(1_000_000)]   # ~8
МБ

# generator expression — итератор, 48 байт:
squares_gen = (x**2 for x in range(1_000_000))

# считаем сумму — без создания списка:
total = sum(x**2 for x in range(1_000_000))
print(total)   # 333332833333500000

# max, min, any, all — тоже работают с
генераторами
```

КЕЙС

Сквозной кейс: обработка телеметрии скважин

- Задача: получить список скважин с аномальным давлением.
- Данные: список словарей с показателями телеметрии.
- Используем: for + enumerate + аккумулятор + break.
- Соберём всё изученное в один рабочий фрагмент.

код

```
telemetry = [
    {"id": "K-12", "P": 18.5},
    {"id": "K-13", "P": 4.2},
    {"id": "K-14", "P": 52.1},
]

anomalies = []
for i, rec in enumerate(telemetry, start=1):
    p = rec["P"]
    if not (5 <= p <= 30):
        anomalies.append(f"#{i} {rec["id"]}:
P={p}")

if not anomalies:
    print("Аномалий не обнаружено")
else:
    print(f"Найдено аномалий: {len(anomalies)}")
    for a in anomalies: print(a)
```

Предскажите результат

1. for i in range(3): print(i) — что выведет?
2. for i in range(10, 0, -3): print(i)
3. x = 0; for i in range(5): x += i; print(x)
4. for i, ch in enumerate('АБВ'): print(i, ch)
5. a=[1,2,3]; b=[10,20]; list(zip(a,b))
6. [x**2 for x in range(4) if x > 0]

ответы

```
1. 0 1 2    (range(3) = 0,1,2)
2. 10 7 4 1  (шаг -3)
3. 10        (0+1+2+3+4 = 10)
4. 0 А / 1 Б / 2 В
5. [(1,10), (2,20)]  (до короткого)
6. [1, 4, 9]  (0 пропущен, x>0)
```

Найдите и исправьте ошибку

1. while i < 10: print(i) — забыт i += 1
2. for i in range(1,10,0): — шаг 0 недопустим
3. for i, j in zip(a): — zip нужно 2+ аргумента
4. for x in 5: — int не итерируемый (нужен range)
5. for i in range(len(wells)): wells[i] — работает,

но enumerate лучше

исправления

```
1. while i < 10: print(i); i += 1
   # забыт инкремент — бесконечный цикл!

2. for i in range(1, 10):  # шаг по умолчанию = 1
   # range не принимает step=0

3. for i, j in zip(a, b):  # два аргумента

4. for x in range(5):  # int → range
   # или for x in [5]:

5. for i, w in enumerate(wells): w
   # pythonic-стиль вместо range(len)
```

Типичные ошибки и ловушки

Бесконечный while забыли i += 1 или изменение условия → зависание.

range(len(x)) антипаттерн — используйте enumerate().

Изменение списка в цикле for x in lst: lst.remove(x) — пропуски!

zip разной длины обрезает по короткому — можно потерять данные.

continue в начале если после него есть важный код — он не выполнится.

Итератор дважды list(it); list(it) → второй раз пусто.

Итоги лекции

1. for — итерация по последовательностям; range() — числовые ряды.
2. while — повторение по условию; осторожно с бесконечным циклом.
3. break — выход; continue — пропуск; else — «цикл без break».
4. enumerate() — индекс+значение; zip() — параллельная итерация.
5. Итераторы — ленивые вычисления; файлы и range — итераторы.
6. Генераторы списков и выражений — коротко и эффективно.

Вопрос 1 из 6

Чем for отличается от while? В каком случае вы выберете while?

Вопрос 2 из 6

Что выведет: for i in range(2, 10, 3): print(i)? Объясните шаг.

Вопрос 3 из 6

Что произойдёт, если в while забыть обновление переменной условия?

Вопрос 4 из 6

Чем break отличается от continue? Что делает else после цикла?

Вопрос 5 из 6

Почему enumerate(wells) лучше, чем for i in range(len(wells))?

Вопрос 6 из 6

Что выведет list(zip([1,2,3], [10,20]))? Почему только две пары?